ITC327W — Work-Integrated Learning

Initial Software Requirements Specification (SRS)

Student Accommodation Payment & Management System — Prototype

Group name:	Group C
Group members:	1. FB Amatebelle 2. Gongxeka IQ 3. Leeuw GA 4. Mohapi MA 5. Molelekeng KP 6. Seleke TM 7. Tshitangano TD
Stakeholder:	Gladys — owner/manager of 3 student accommodation properties
Date:	27/08/2026

1. Project Introduction

This project proposes an integrated prototype to help small student-accommodation providers manage rental payments, tenant information, maintenance requests and communication in one system, replacing manual spreadsheet and WhatsApp-based processes. The stakeholder for this project is Gladys Belle, who manages three student accommodation properties. The prototype consists of a Flutter mobile application for students, an ASP.NET web application for management, and a shared Supabase backend.

2. Stakeholder Profile and Current Process

2.1 Stakeholder Profile

Organisation	Graceful Plans
Organisation type	Small private student-accommodation provider (3 properties)
Stakeholder	Gladys Belle: Owner/manager
Main activities	Renting rooms to students, collecting rent, coordinating maintenance, communicating with tenants
Reason selected	Real, accessible stakeholder with a genuine manual-process problem across payments, tenant records and maintenance

2.2 Current Process

Gladys currently manages student accommodation information using an Excel spreadsheet. When a new student joins, their name, contact details, agreed rental amount and other relevant information are recorded after the contract is signed. Rental payments are manually recorded when received.

Day-to-day communication happens mainly through WhatsApp, since Gladys cannot regularly visit the properties in person. Students report problems and request assistance through WhatsApp. When maintenance is required, Gladys contacts a technician directly; physical site visits happen only occasionally.

2.3 Problems Experienced

- Tracking rental payments is the most time-consuming part of the process, taking approximately four hours and often done late at night.
- Students or their parents sometimes use incorrect or unclear payment references, making it difficult to identify who made a payment.
- Proof of payment must sometimes be requested manually, and students may take several days (or longer) to respond.
- Manual tracking can result in unpaid rent being missed , in one case, a student accumulated five months of unpaid rent because it was not followed up in time.
- Uncollected rent affects the business financially; Gladys has sometimes had to use her own salary to cover shortfalls.
- Maintenance requests sent privately through WhatsApp are easy to forget — both for Gladys and for the technician.
- Students have no way of knowing whether a maintenance problem they did not report has already been dealt with.
- There is no centralised way to see, at a glance, outstanding payments or outstanding maintenance tasks.

3. Problem Statement

Gladys currently uses an Excel spreadsheet and WhatsApp to manage rental payments, tenant records, and maintenance requests across three student accommodation properties. This results in payment tracking that takes approximately four hours per session, unpaid rent that can go unnoticed for months due to unclear payment references, and maintenance requests that are easily forgotten. The problem affects both Gladys and her student tenants because there is no organised, shared source of truth for payment status, payment history, or maintenance requests. An improved solution is needed to centralise rental payment tracking, provide clear payment references and history, and give both management and students visibility of maintenance requests and their status.

4. Project Aim and Objectives

Aim

To develop an integrated prototype that simplifies rental payment tracking, tenant record-keeping, and maintenance-request management for a small student-accommodation provider.

Objectives

- Allow students to view their payment reference, rental amount, rental period and payment history.
- Allow management to record and track rental payments against individual students.
- Provide management with a monthly report showing which students have paid and which have not.
- Allow students to submit maintenance requests and view the status of reported requests.
- Allow management to update the status of maintenance requests.
- Restrict access to payment and personal information to the relevant student and management only.

5. Project Scope

Deliverables	Flutter mobile prototype ASP.NET web prototype Supabase backend
Acceptance criteria	Payments recorded in ASP.NET are reflected in the student's Flutter view

	Maintenance requests submitted in Flutter appear in the ASP.NET dashboard and are visible to other students A monthly paid/unpaid report can be generated from recorded payment data
Exclusions	Real online payment processing / payment gateway integration (the system tracks payment status, not live transactions) Automated SMS/WhatsApp reminders (out of scope for the prototype)
Constraints	Must be completed within one semester Group's current technical skill level in Flutter, ASP.NET and Supabase Prototype must use test data rather than real student financial information
Assumptions	Gladys remains available for feedback and testing throughout the semester Three properties' worth of test data is sufficient to demonstrate the prototype

6. Stakeholder Needs

The following needs were identified and confirmed during the stakeholder interview conducted with Gladys (see Appendix B):

- Easier management of student accommodation and rental information.
- Clear, correct payment references for each student to reduce payment-matching errors.
- Rental payment records and a full payment history per student.
- A monthly report identifying which students have paid and which have not.
- A way to submit, view and track maintenance requests, including outstanding vs. completed status.
- Shared visibility of maintenance requests among students, so duplicate reports are avoided.
- Strict privacy of payment and personal information — visible only to the relevant student and management.
- OTP-based authentication rather than shared passwords.
- A system simple enough to use with limited technological experience, given appropriate training.

7. Functional Requirements

ID	Requirement	Type	Priority	Related stakeholder need
FR1	The student must be able to view their unique payment reference in the Flutter application.	Functional	Must Have	Correct payment references
FR2	The student must be able to view their rental amount, rental period and payment history.	Functional	Must Have	Payment history visibility
FR3	Management must be able to record a payment received against a specific student's account.	Functional	Must Have	Easier payment tracking
FR4	The system must associate each recorded payment with a student via their assigned payment reference.	Functional	Must Have	Difficulty matching payments
FR5	Management must be able to generate a monthly report showing which students have paid and which have not.	Functional	Must Have	Monthly payment reporting
FR6	The student must be able to submit a maintenance request describing the problem.	Functional	Must Have	Maintenance tracking

FR7	All students must be able to view submitted maintenance requests and their current status.	Functional	Must Have	Shared maintenance visibility
FR8	Management must be able to update the status of a maintenance request (outstanding / resolved).	Functional	Must Have	Outstanding task visibility
FR9	Management must be able to view a dashboard summarising outstanding payments and outstanding maintenance tasks.	Functional	Should Have	Centralised visibility
FR10	Management must be able to add and manage tenant records (name, contact details, room, rental period).	Functional	Must Have	Manual Excel record-keeping
FR11	Management must be able to post announcements visible to students.	Functional	Could Have	General communication needs

8. Non-Functional Requirements

ID	Requirement	Type	Priority	Related stakeholder need
NFR1	Users must authenticate using an OTP rather than a shared password.	Non-functional	Must Have	OTP over shared passwords
NFR2	Payment and personal information must be visible only to the relevant student and management, not to other students.	Non-functional	Must Have	Privacy of payment info
NFR3	The interface must be simple and understandable for a user with limited technological experience.	Non-functional	Must Have	Limited tech experience
NFR4	The prototype must use test data rather than real student financial or personal information.	Non-functional	Must Have	Responsible data handling
NFR5	Error messages must be clear and understandable to a non-technical user.	Non-functional	Should Have	Ease of use

9. System Requirements

ID	Requirement	Type	Priority	Related stakeholder need
SR1	The mobile application must be developed using Flutter.	System	Must Have	Module requirement
SR2	The web application must be developed using ASP.NET.	System	Must Have	Module requirement
SR3	The shared backend and database must use Supabase.	System	Must Have	Module requirement

10. Integration Requirements

ID	Requirement	Type	Priority	Related stakeholder need

IR1	Payments recorded through the ASP.NET application must be stored in Supabase and reflected in the student's Flutter view.	Integration	Must Have	Shared payment status
IR2	Maintenance requests submitted through Flutter must be stored in Supabase and displayed in the ASP.NET dashboard.	Integration	Must Have	Shared maintenance tracking
IR3	Updates to maintenance-request status made in ASP.NET must be reflected in the Flutter application.	Integration	Must Have	Status visibility for students

11. Requirements Change Log

Date	Change	Reason
28/08/2026	Refined the payment requirement from a general "view rent status" function into an auto-matched payment reference and monthly paid/unpaid report.	Interview with Gladys identified unclear payment references and manual reference-matching as her single biggest pain point.
28/08/2026	Changed maintenance-request visibility from private (student-to-management only) to shared/general among students.	Gladys explained that shared visibility prevents duplicate reports and lets students see if an issue is already being handled.
28/08/2026	Selected OTP-based authentication instead of biometric login.	Biometric access is not available on every device; OTP was agreed as a workable alternative during the interview.

12. Refined Project Scope

Based on the stakeholder interview, the project scope taken forward into system design centres on three core areas: (1) rental payment recording, reference-matching and monthly reporting; (2) tenant record management; and (3) maintenance-request submission, shared visibility and status tracking all secured through OTP-based authentication and strict access control over payment information. Real online payment processing remains excluded; the system tracks payment status rather than handling live transactions.

13. System Architecture and Design

This section continues the Software Requirements Specification submitted in Phase 1. It turns the approved Phase 1 requirements into a system design for StayTrack. The design shows how the Flutter mobile application, the ASP.NET web application and the shared Supabase backend operate as one integrated system. It is based on the stakeholder requirements and refined project scope confirmed in Phase 1, together with the design changes recorded in Section 13.1.

13.1 Changes Since Phase 1

While designing the system, the group added the following to the approved Phase 1 baseline. Each change responds to a problem the stakeholder described in the Phase 1 interview.

Change	Reason
Added a Maintenance Technician role with a	The stakeholder explained that maintenance problems are forgotten by both herself and the

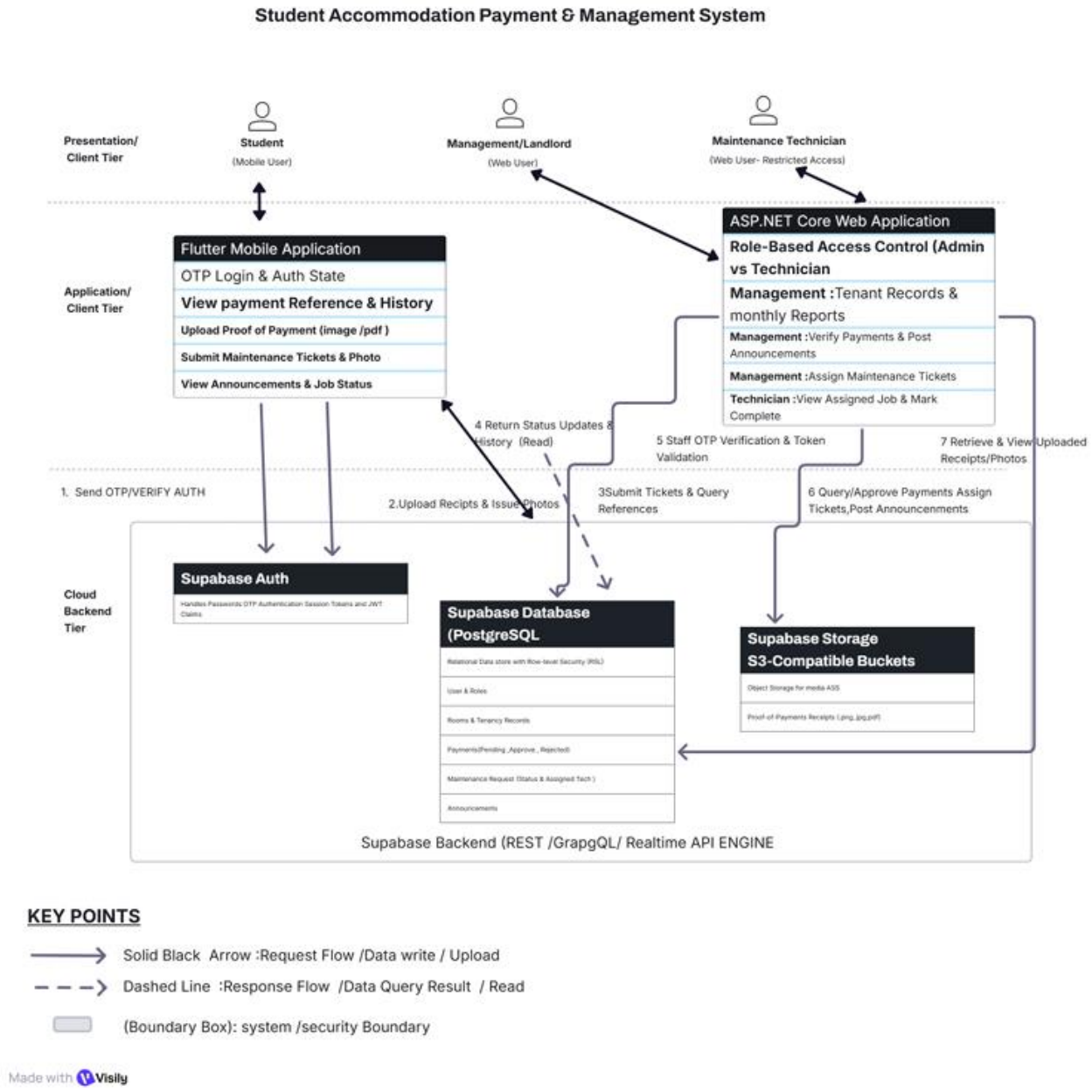
restricted view of assigned requests	technician. Assigning requests and recording completion makes outstanding work visible.
Added student upload of proof of payment, verified by management (Supabase Storage)	The stakeholder currently has to ask students for proof of payment, which can take several days or receive no response.
Added an optional photo on maintenance requests (Supabase Storage)	A photo helps the technician understand the problem before attending.
Monthly paid/unpaid report is generated on demand and not stored	The interview asked for an end-of-month report showing who has paid. No archive of old reports was requested, so storing them is not needed for the prototype.
Added a property table, a unique payment reference per tenant, and lease dates per tenant to the database design	The stakeholder manages several properties, identified unclear payment references as her biggest problem, and explained that students have different rental periods. These refine Phase 1 requirements and add no new features.

The changes add the following requirements to the Phase 1 requirements tables. The numbers continue the Phase 1 sequence.

ID	Requirement	Type	Priority
FR12	The student must be able to upload proof of payment for a rental payment through the Flutter application.	Functional	Must Have
FR13	Management must be able to approve or reject a payment after reviewing the proof of payment.	Functional	Must Have
FR14	The student must be able to attach an optional photo to a maintenance request.	Functional	Should Have
FR15	Management must be able to assign a maintenance request to a maintenance technician.	Functional	Must Have
FR16	The maintenance technician must be able to view the requests assigned to them and mark a completed job as completed.	Functional	Must Have
SR4	Proof-of-payment files and maintenance photos must be stored in Supabase Storage.	System	Must Have
IR4	Files uploaded through Flutter must be viewable through the ASP.NET application.	Integration	Must Have

13.2 System Architecture

StayTrack has three user roles: the student, the accommodation manager (management) and the maintenance technician. Students use the Flutter mobile application. Management and the technician use the same ASP.NET web application, and each role sees only the pages it is permitted to use. Both applications connect to one Supabase backend made up of Authentication, Database and Storage.



13.3 UML Design

13.3.1 Use Case Diagram

The use case diagram shows the three actors that interact with StayTrack and the functions each can perform inside the system boundary.

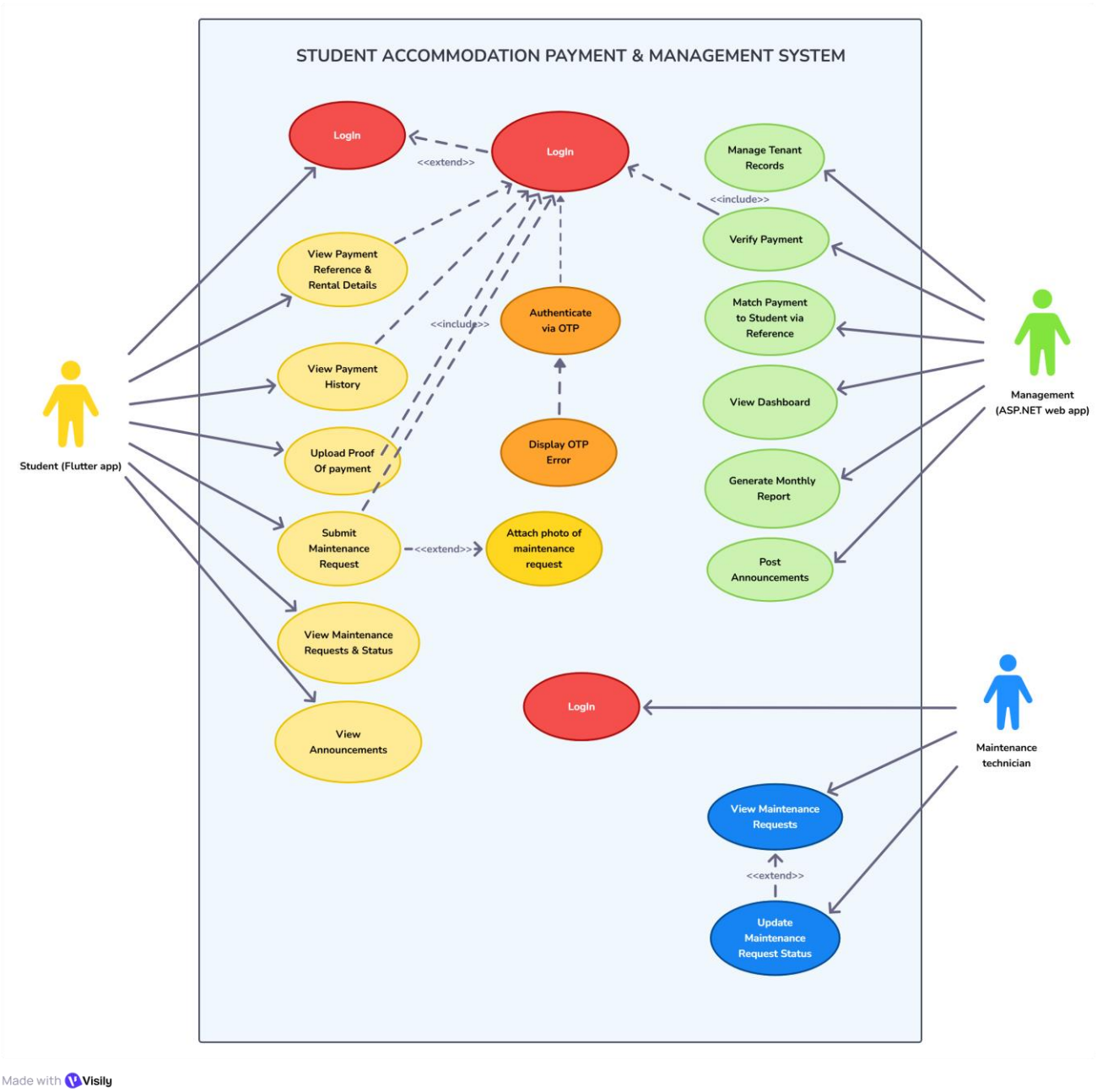


Figure 2: StayTrack use case diagram

Actor	What the actor needs to do
Student	Log in with an OTP; view their payment reference and payment history; upload proof of payment; submit a maintenance request with an optional photo; view the status of maintenance requests; view announcements.
Management	Log in with an OTP; manage tenant records; verify payments; generate the monthly report; assign maintenance requests to a technician; post announcements; view the dashboard of outstanding payments and tasks.
Maintenance Technician	Log in with an OTP; view the maintenance requests assigned to them; approve a completed job.

Relationships used. Each actor is connected to its use cases by an association. `Login` is included by Upload Proof of Payment, Submit Maintenance Request and Verify Payment, because these functions cannot be performed without logging in. Display OTP Error extends Login, because the error appears only when the OTP entered is invalid. Attach Photo to Maintenance Request extends Submit Maintenance Request, because the photo is optional and is added only in some cases. Generalization is not used, because Management and the

Maintenance Technician have different functions and share no behaviour that would justify a more general actor.

13.3.2 Sequence Diagrams

Two sequence diagrams show important processes: one through the Flutter application and one through the ASP.NET application. Each represents a use case on the use case diagram.

Sequence Diagram 1: Student submits a maintenance request (Flutter)

Related use case and requirements: Submit Maintenance Request and Attach Photo to Maintenance Request; FR6, FR14, IR2.

Participants: Student, Flutter application, Supabase Storage, Supabase Database.

1. The student enters the title and description in the Flutter app and optionally attaches a photo.
2. Flutter checks the entered details.
3. If a photo is attached, Flutter uploads it to Supabase Storage, which returns the file path.
4. Flutter sends the request (title, description, photo path and the status "pending") to the Supabase Database.
5. Supabase stores the record and returns a confirmation.
6. Flutter shows a confirmation to the student.
7. The request is then visible in the maintenance list for students and in the ASP.NET application for management.

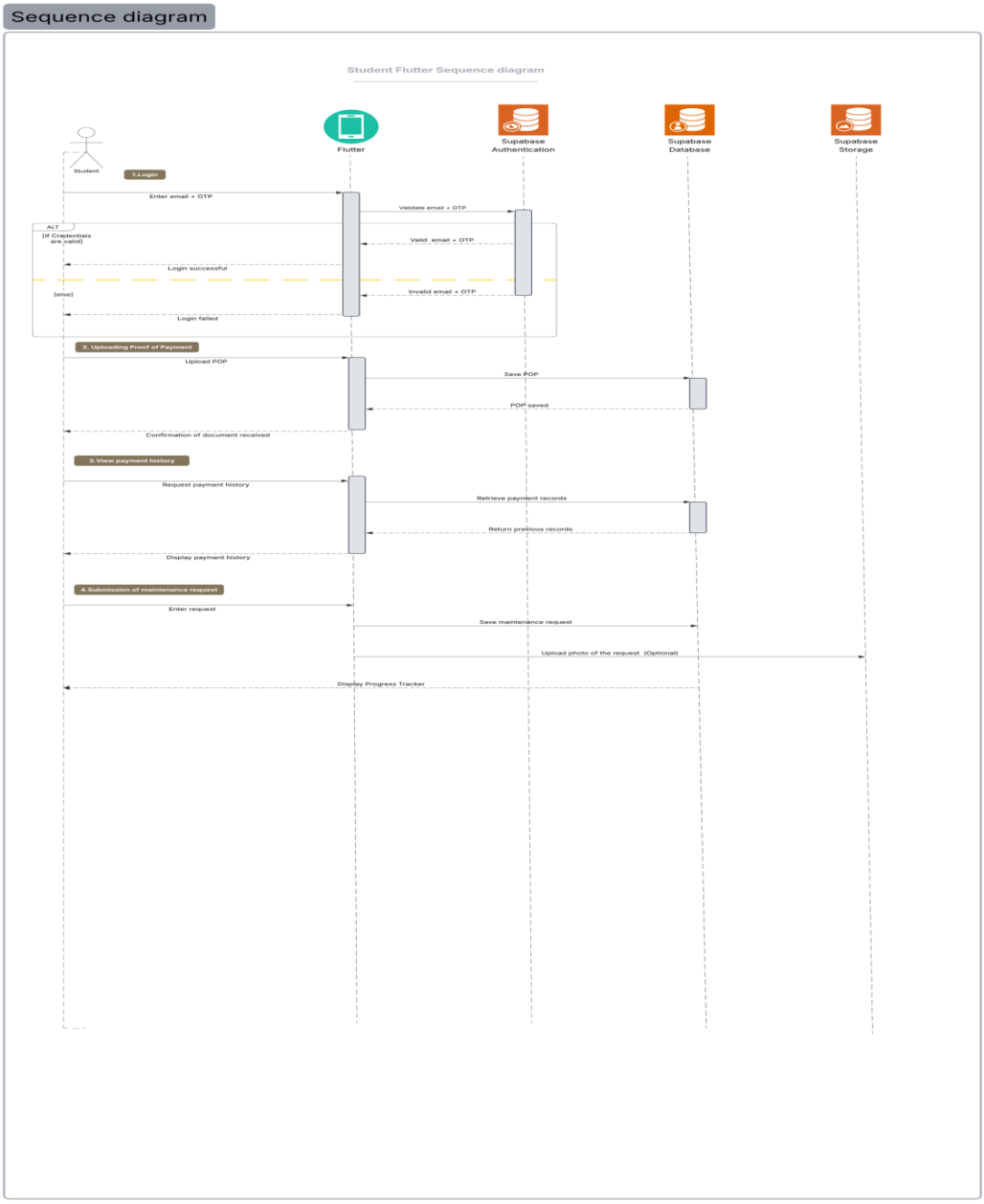


Figure 3: Sequence diagram: student submits a maintenance request

Sequence Diagram 2: Management verifies a payment (ASP.NET)

Related use case and requirements: Verify Payment; FR13, IR1, IR4.

Participants: Management, ASP.NET web application, Supabase Database, Supabase Storage.

1. Management opens the payments page in the ASP.NET application.
2. ASP.NET requests the pending payments from the Supabase Database, which returns the list.
3. Management selects a payment. ASP.NET retrieves the proof-of-payment file from Supabase Storage and displays it.
4. Management approves or rejects the payment.
5. ASP.NET updates the payment status, the verifier and the verification time in the Supabase Database, which returns a confirmation.
6. ASP.NET shows the updated status.
7. The student then sees the updated payment status in the Flutter application.

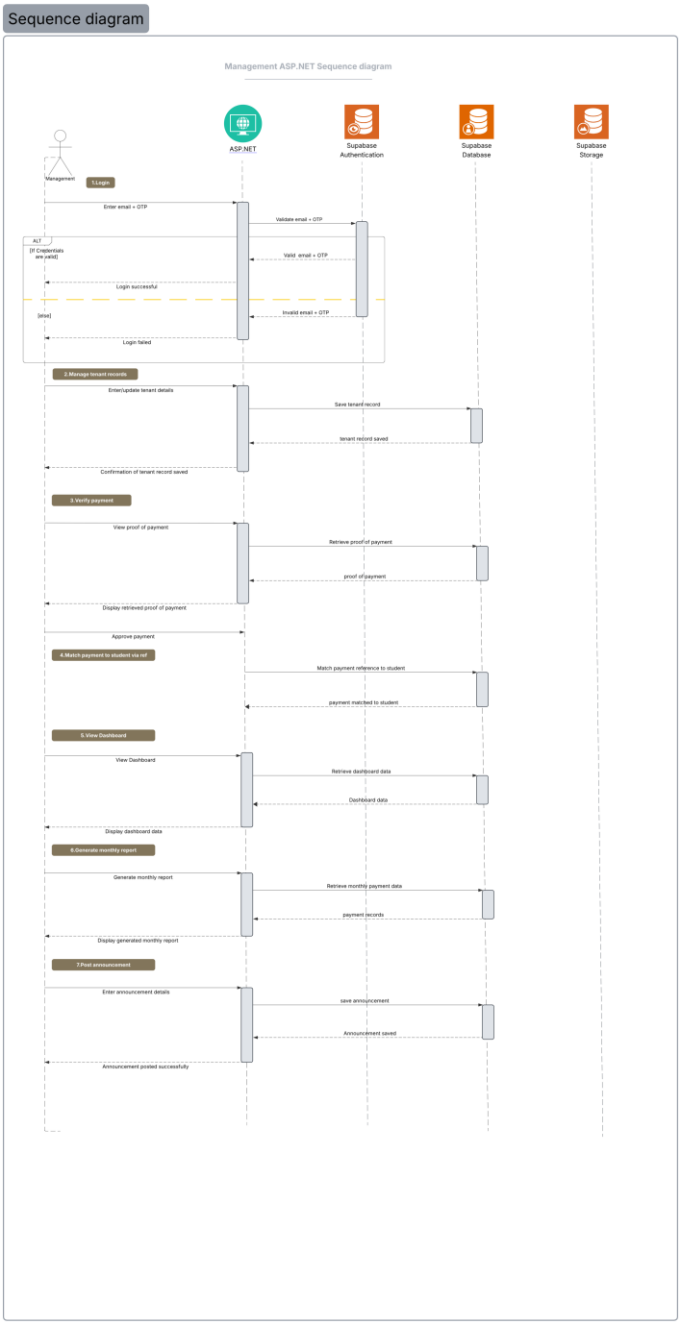


Figure 4: Sequence diagram: management verifies a payment

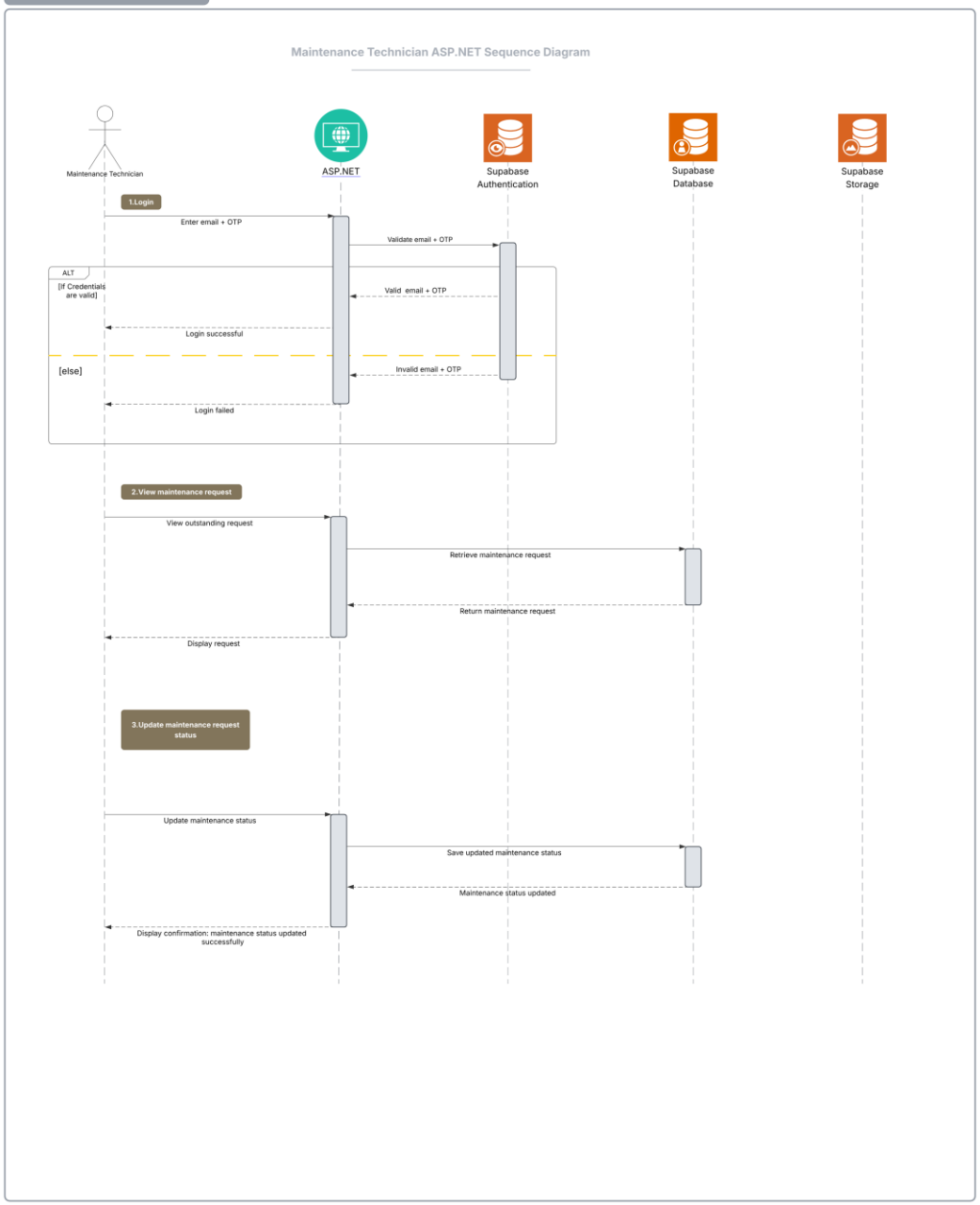
Sequence Diagram 3 (include only if your group created it): User logs in with an OTP

Related use case and requirement: Login; NFR1.

Participants: User, application (Flutter or ASP.NET), Supabase Authentication.

1. The user enters their email address in the application.
2. The application asks Supabase Authentication to send an OTP.
3. Supabase Authentication sends the OTP to the user's email address.
4. The user enters the OTP in the application.
5. The application sends the OTP to Supabase Authentication for verification.
6. Supabase Authentication returns a session token and the user's role, and the application opens the screens for that role.

Sequence diagram



13.4 Database Design (ERD)

The Supabase database has nine tables. They hold the information that the Flutter and ASP.NET interfaces capture and display. Authentication is handled by Supabase Authentication, so the design stores no OTP codes. Uploaded files are held in Supabase Storage, and the database stores only the file path. Monthly reports are generated on demand, so there is no report table.

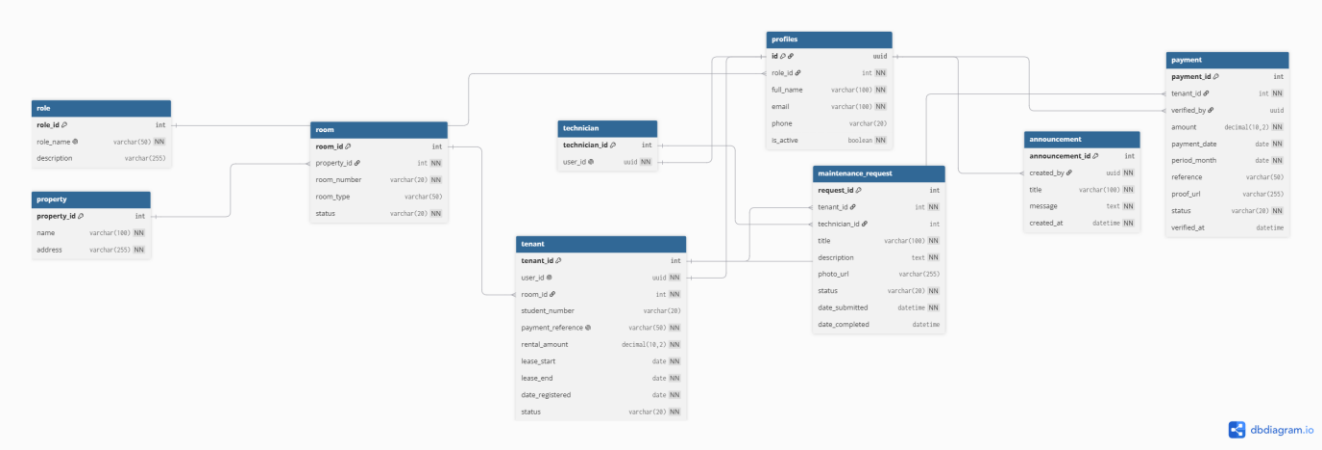


Figure 6: StayTrack entity relationship diagram

Table	Purpose	Primary key	Foreign keys
role	The three user roles (student, management, technician)	role_id	None
profiles	One row for each user, linked to the Supabase Authentication user	id	role_id
property	The accommodation properties	property_id	None
room	The rooms in each property	room_id	property_id
tenant	A student's tenancy: room, unique payment reference, rental amount and lease dates	tenant_id	user_id, room_id
technician	A maintenance technician, linked to a user	technician_id	user_id
payment	A student's payment, its proof of payment and its verification status	payment_id	tenant_id, verified_by
maintenance_request	A maintenance request, its optional photo, its status and the assigned technician	request_id	tenant_id, technician_id
announcement	An announcement posted by management	announcement_id	created_by

From	To	Cardinality	Meaning
role	profiles	1 to many	A role is held by many users.
profiles	tenant	1 to 1	A student user has one tenancy record.
profiles	technician	1 to 1	A technician user has one technician record.
profiles	announcement	1 to many	A manager can post many announcements.
profiles	payment	1 to many	A manager can verify many payments.
property	room	1 to many	A property has many rooms.
room	tenant	1 to many	A room can be linked to many tenants.

tenant	payment	1 to many	A tenant makes many payments.
tenant	maintenance_request	1 to many	A tenant submits many maintenance requests.
technician	maintenance_request	1 to many	A technician is assigned many requests.

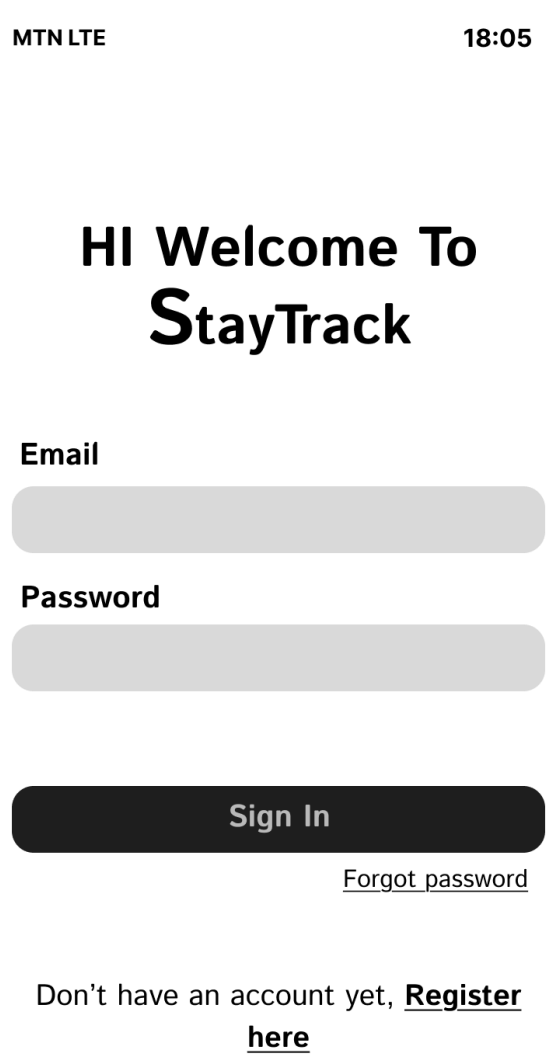
Status values. A payment has the status pending, approved or rejected. A maintenance request has the status pending, assigned or completed.

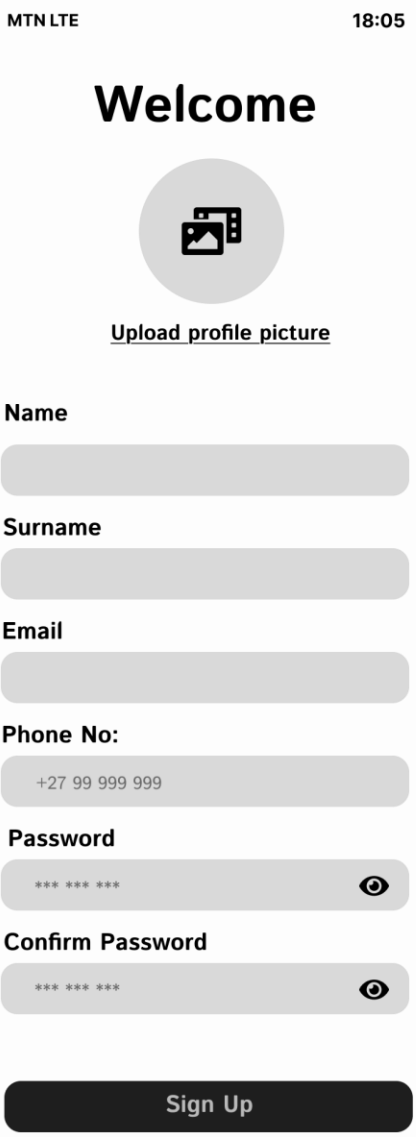
Fields added to support Phase 1 requirements. `tenant.payment_reference` is unique for each student and lets payments be matched to the right student (FR1, FR4). `payment.period_month` records which month a payment covers, which the monthly paid/unpaid report needs (FR5). `payment.verified_by` and `payment.verified_at` record who verified a payment and when (FR13). The `property` table is included because the stakeholder manages several properties.

13.5 User Interface Design

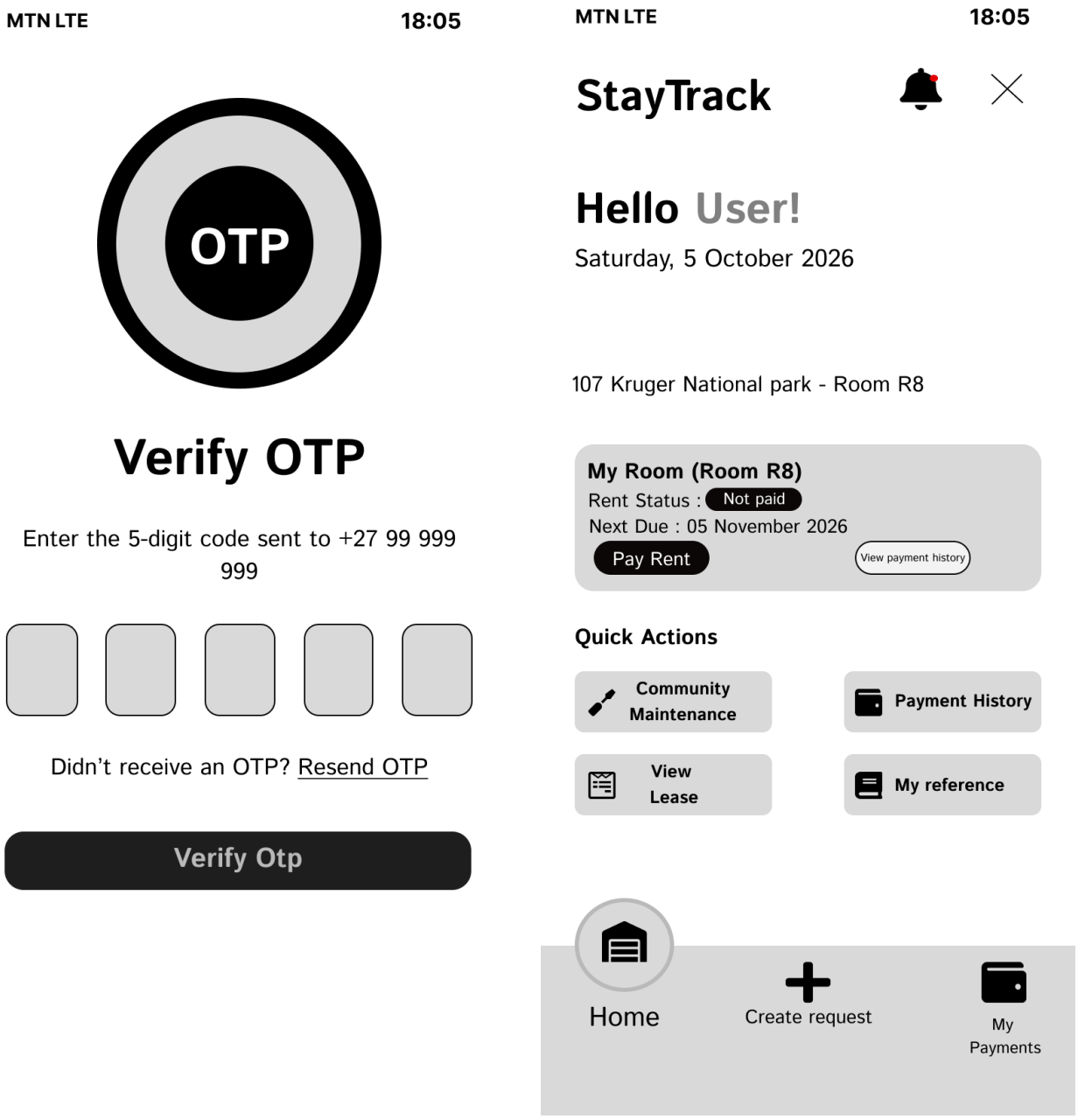
13.5.1 Flutter Mobile Screens

Screen	User	Purpose	Requirement / use case
Sign in / Sign up	Student	Register and enter an email address to receive an OTP	NFR1; Login
OTP Verification	Student	Verify the user's identity with the OTP	NFR1; Login
Home	Student	Show payment status, payment reference, quick actions and the latest announcement	FR1, FR2, FR11
My Payments	Student	Show payment reference, rental amount, lease period and payment history	FR1, FR2; View Payment Reference & History
Payment Detail	Student	Show one payment, its status and its proof of payment	FR2, FR12
Upload Proof of Payment	Student	Capture the month paid for, amount, date, reference and the proof file	FR12; Upload Proof of Payment
Maintenance Requests	Student	List all requests with their status	FR7; View Maintenance Request Status
Submit Maintenance Request	Student	Capture the title, description and an optional photo	FR6, FR14; Submit Maintenance Request
Request Detail	Student	Show one request, its photo and its status	FR7
Announcements	Student	Show announcements posted by management	FR11; View Announcements
Profile	Student	Show the student's details, room and lease dates	FR2

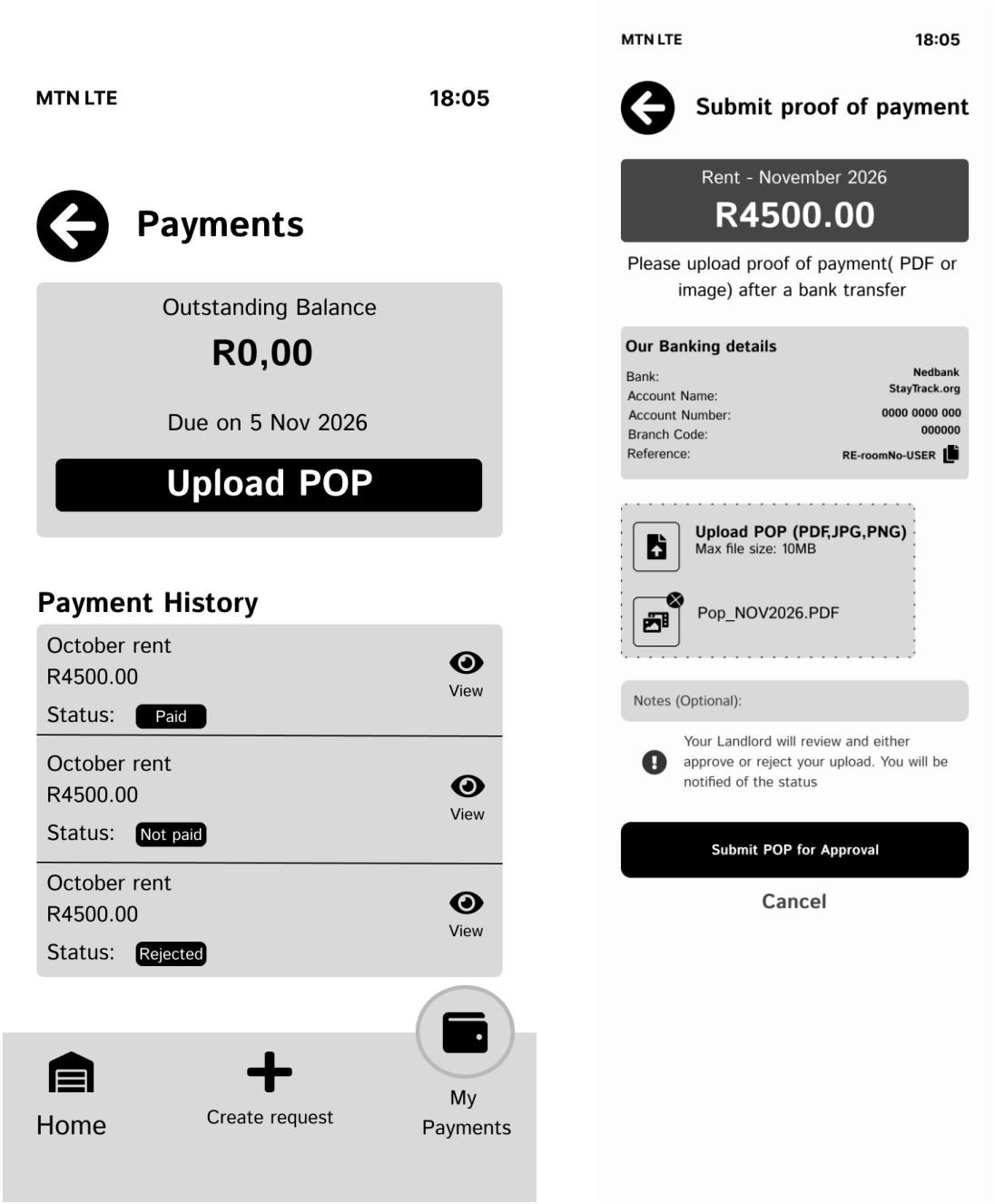

MTN LTE
18:05
HI Welcome To StayTrack
Email
Password
Sign In
Forgot password
Don't have an account yet, Register here


MTN LTE
18:05
Welcome
Upload profile picture
Name
Surname
Email
Phone No:
+27 99 999 999
Password
*** *** ***
Confirm Password
*** *** ***
Sign Up

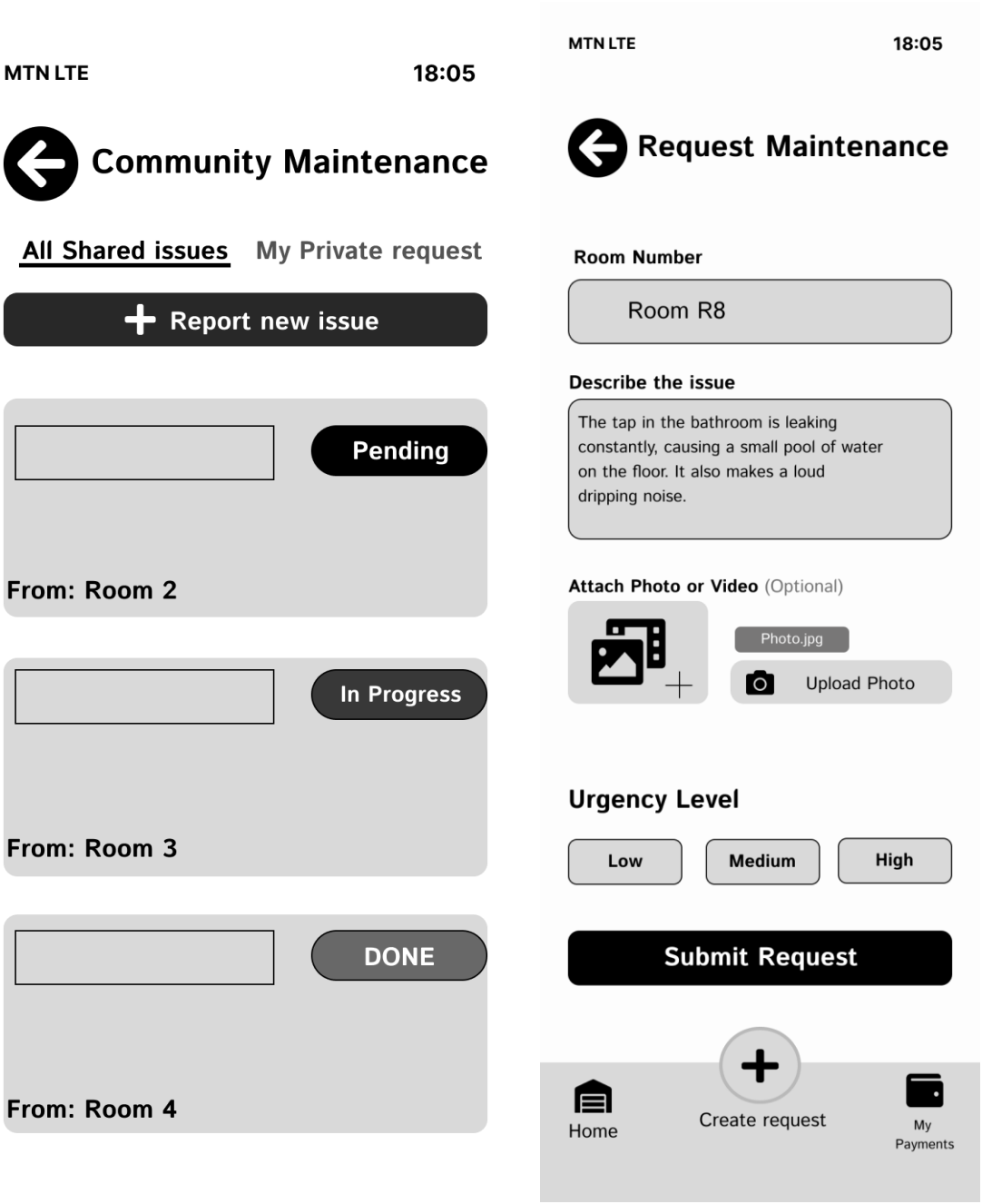
Sign in page



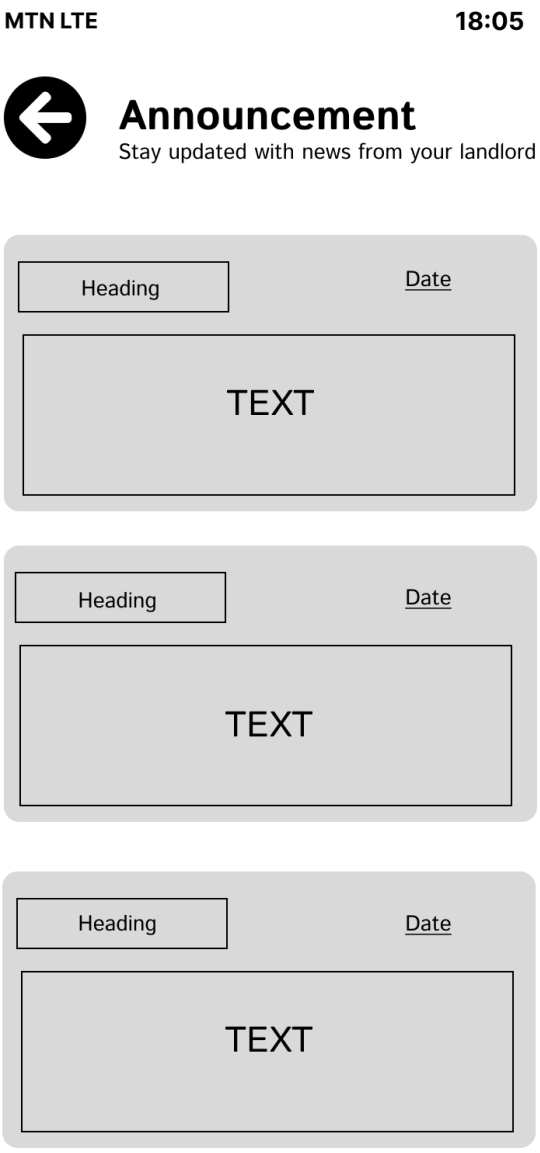
OTP Verification & Home screen



Payment details and upload proof of payment screens



Maintenance request screen (shared) & submit maintenance screen

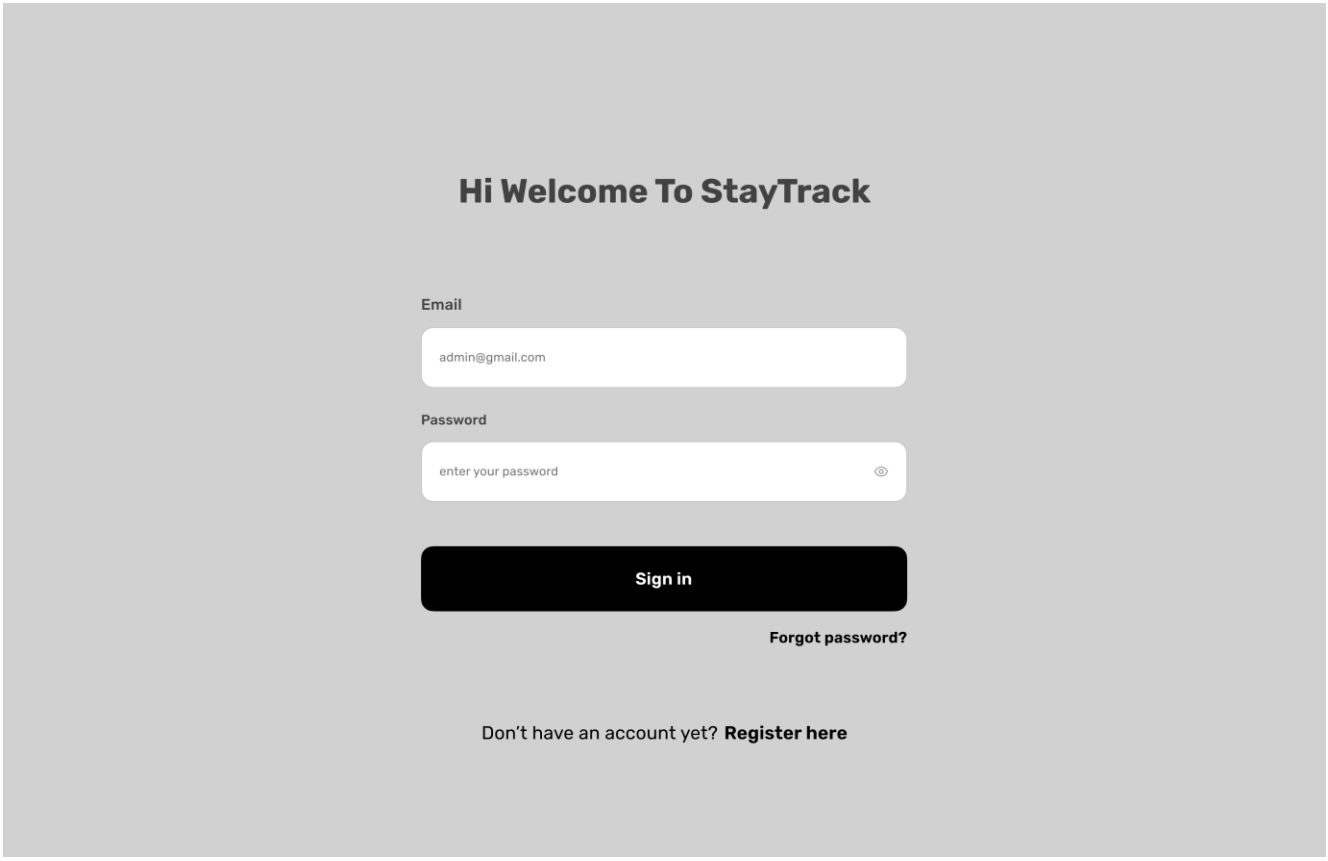


Announcement screen

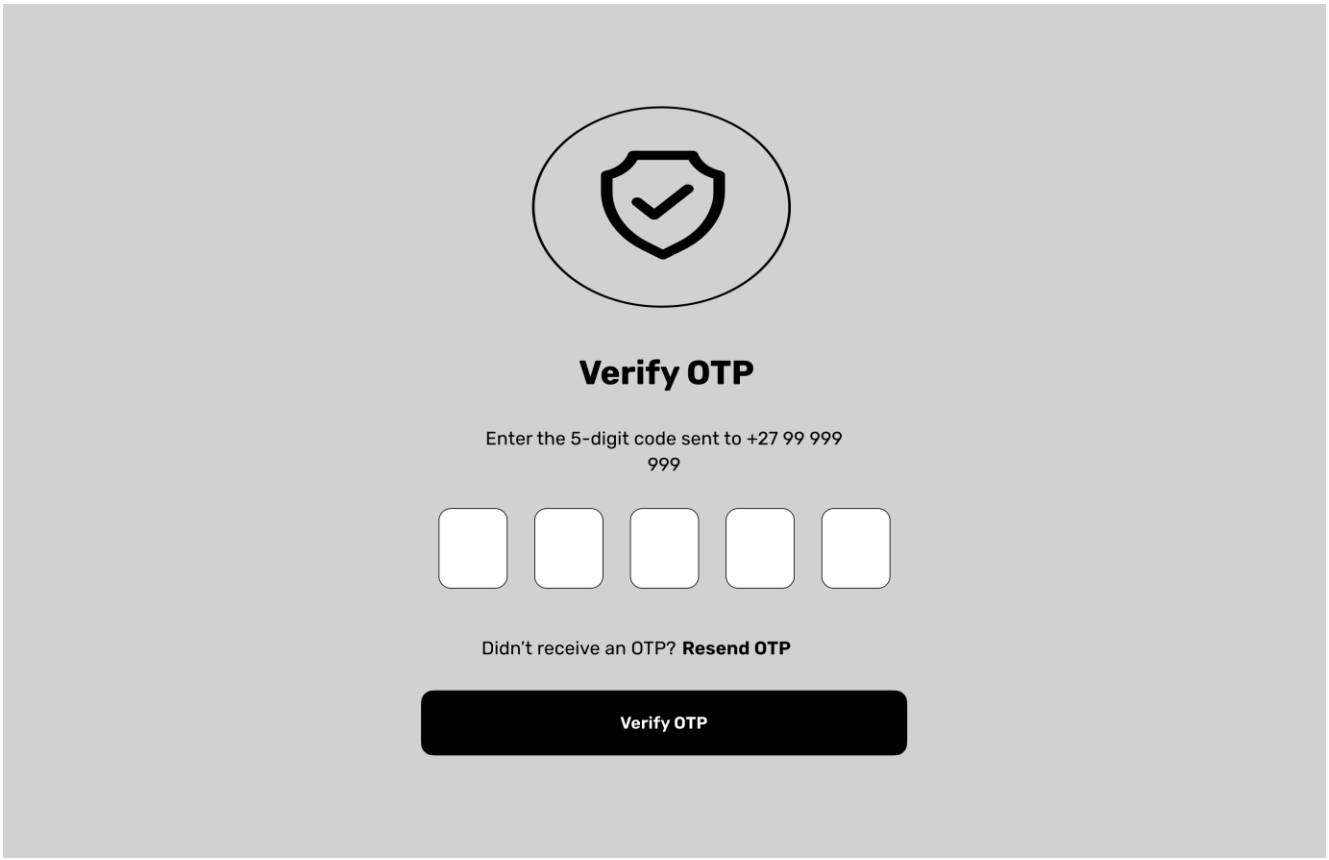
13.5.2 ASP.NET Web Pages

Page	User	Purpose	Requirement / use case
Login (OTP)	Management, Technician	Verify the user and open the pages for their role	NFR1; Login
Dashboard	Management	Summarise outstanding payments and outstanding maintenance tasks	FR9; View Outstanding Payments & Tasks Dashboard
Tenant Management	Management	Add, edit and archive tenant, room and property details	FR10; Manage Tenant Records
Payments	Management	Record payments, review proof of payment, approve or reject	FR3, FR4, FR13; Verify Payment

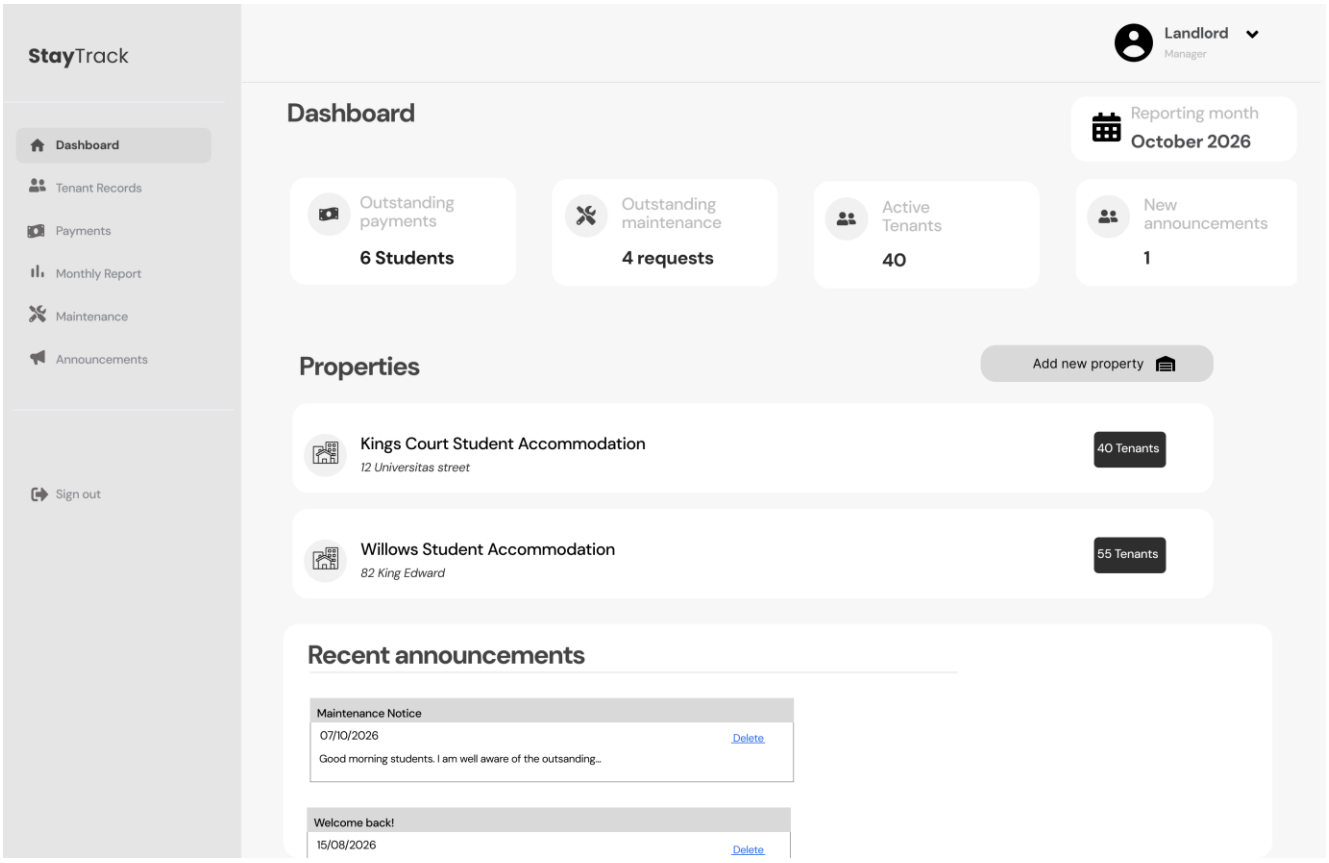
Monthly Report	Management	Choose a month, generate the paid/unpaid report and download it	FR5; Generate Monthly Report
Maintenance Management	Management	View requests, assign a technician and update status	FR8, FR15; Assign Maintenance Request
Post Announcement	Management	Write and post an announcement to students	FR11; Post Announcement
My Assigned Requests	Technician	List the requests assigned to the technician	FR16; View Assigned Maintenance Requests
Request Detail (technician)	Technician	Show a request and its photo and mark the job as completed	FR16; Approve Completed Job



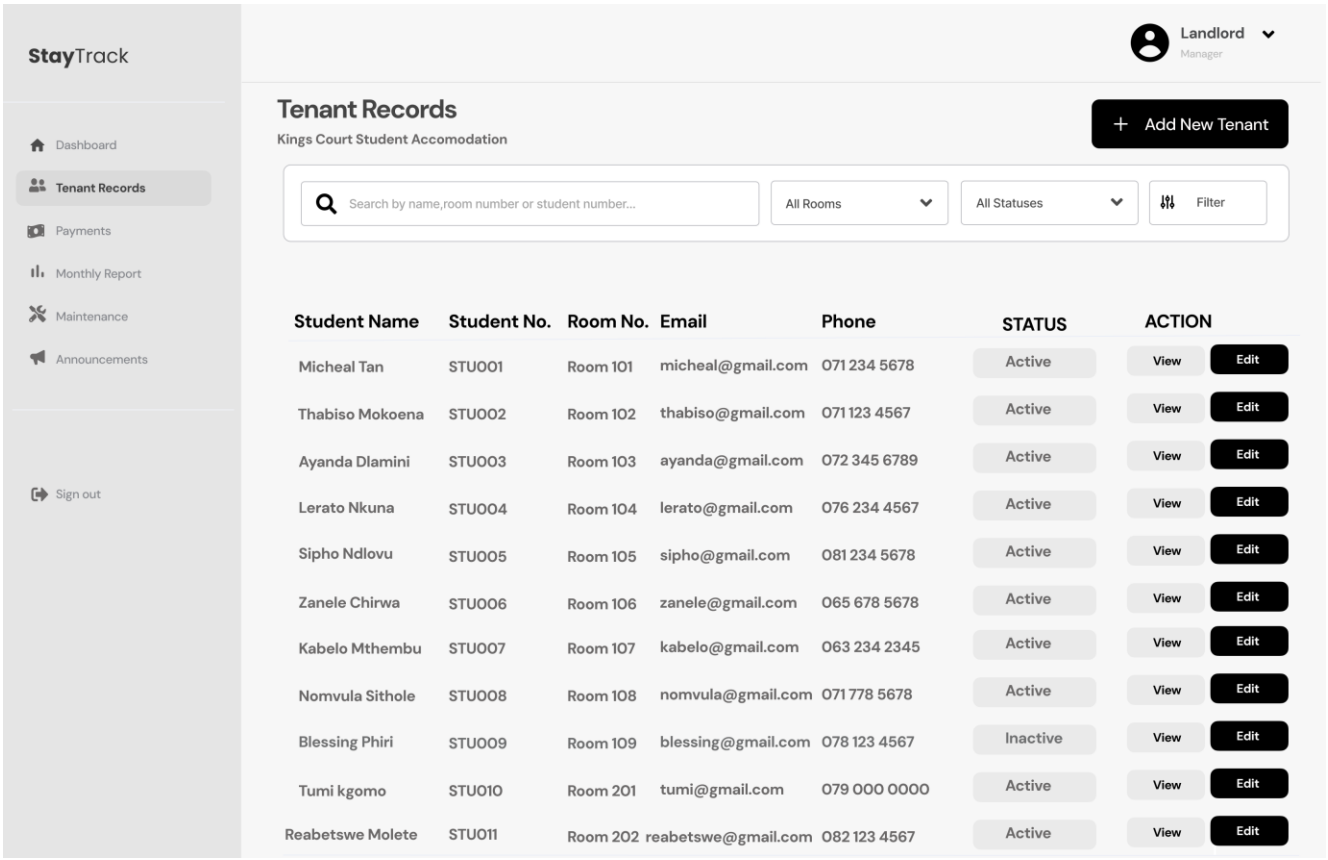
Login page



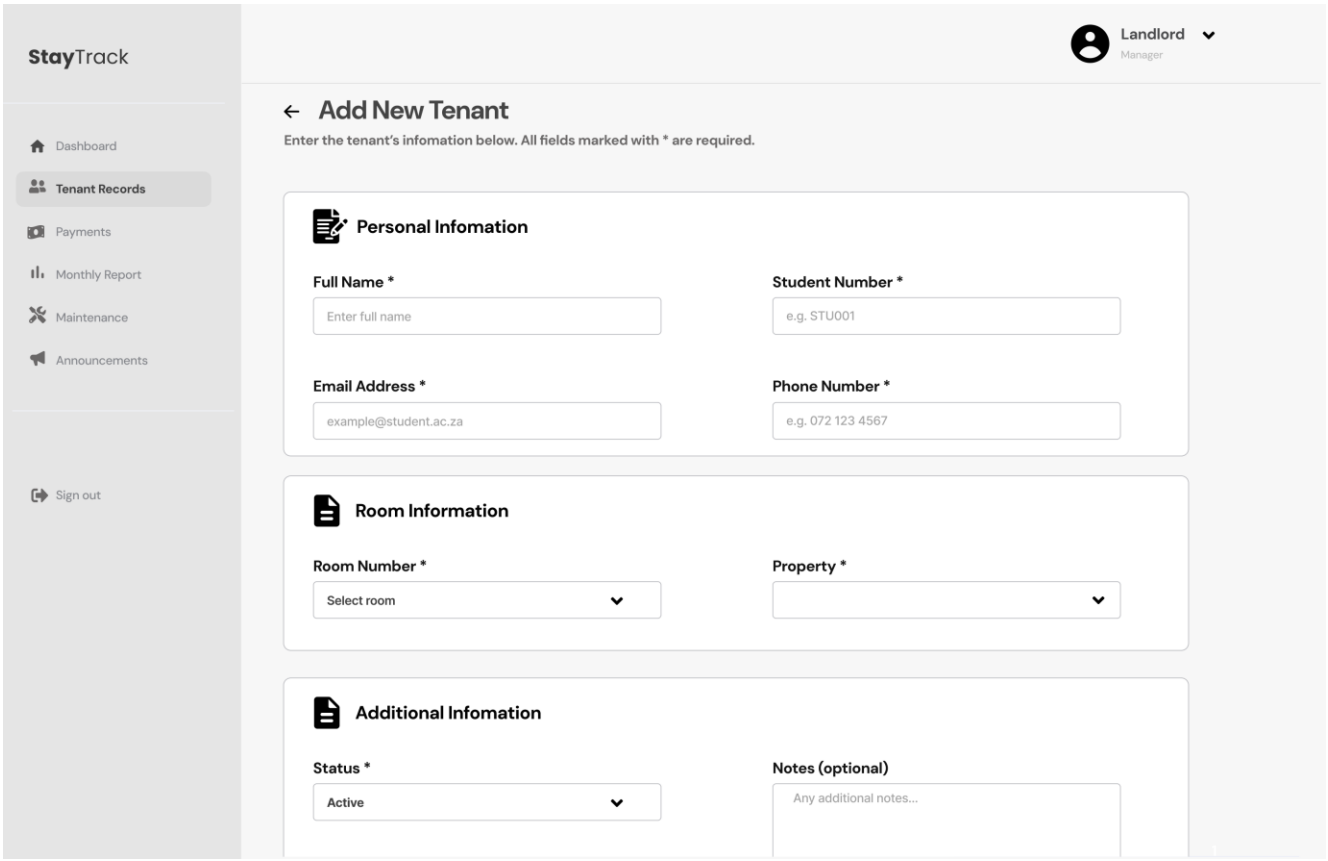
OTP authentication screen



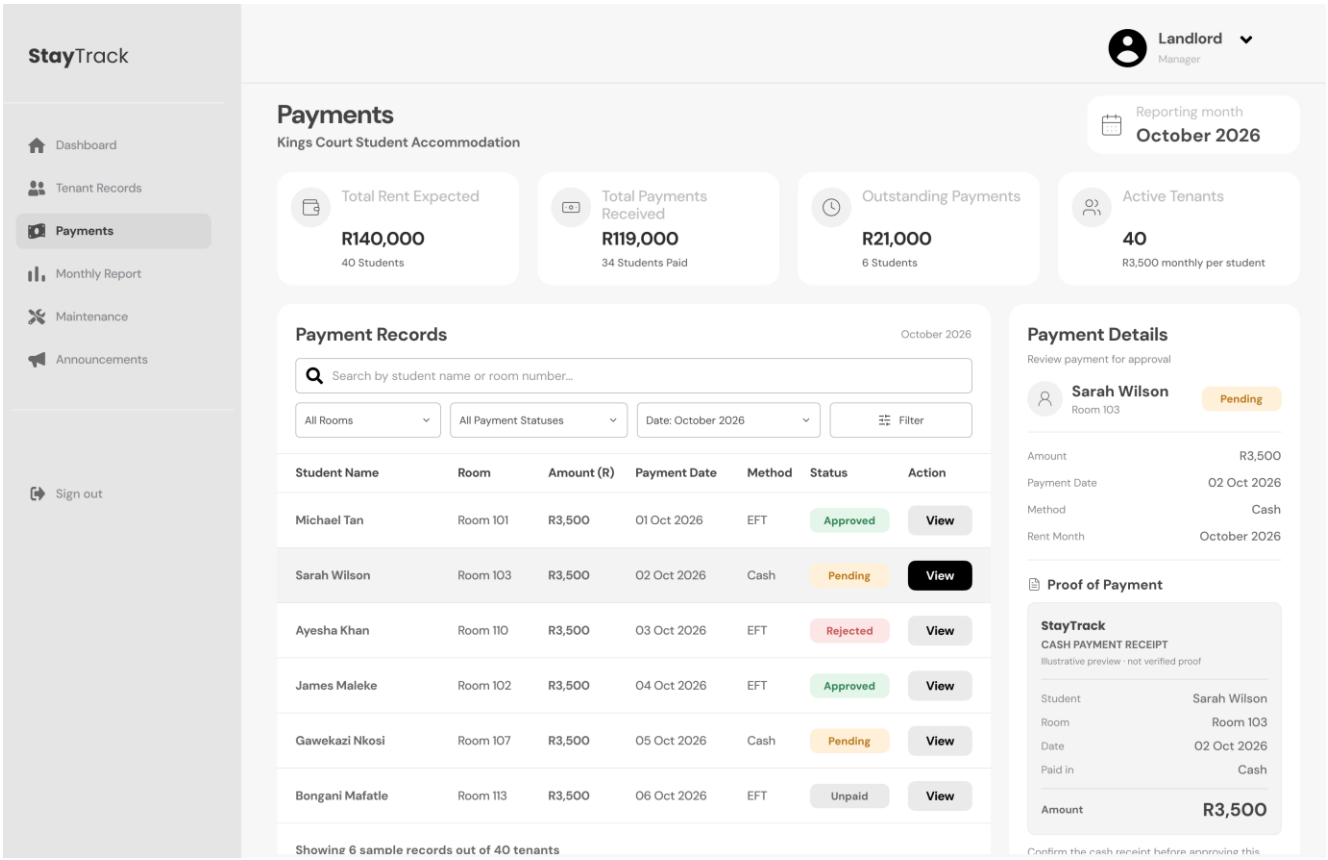
Landlord dashboard



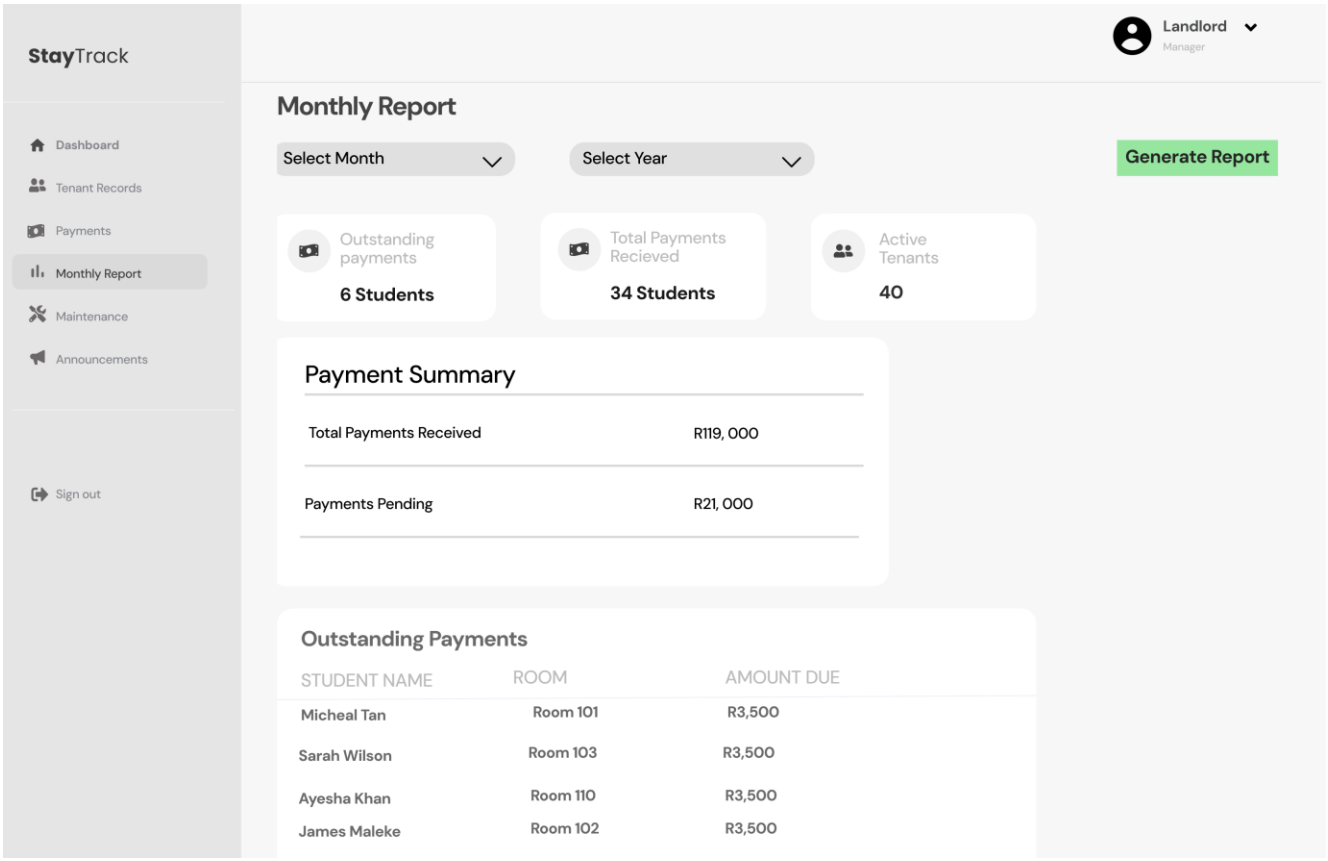
tenant dashboard



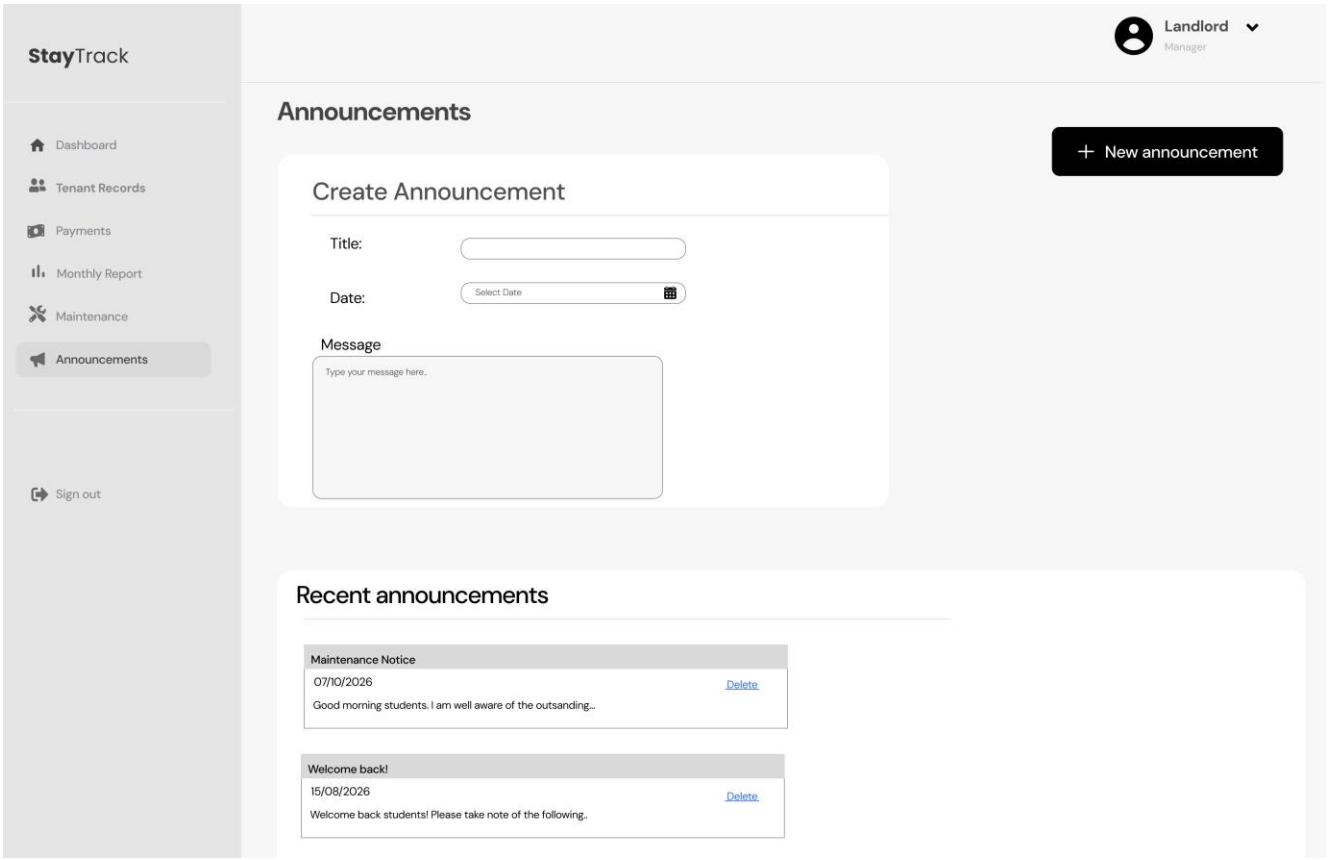
add tenant screen



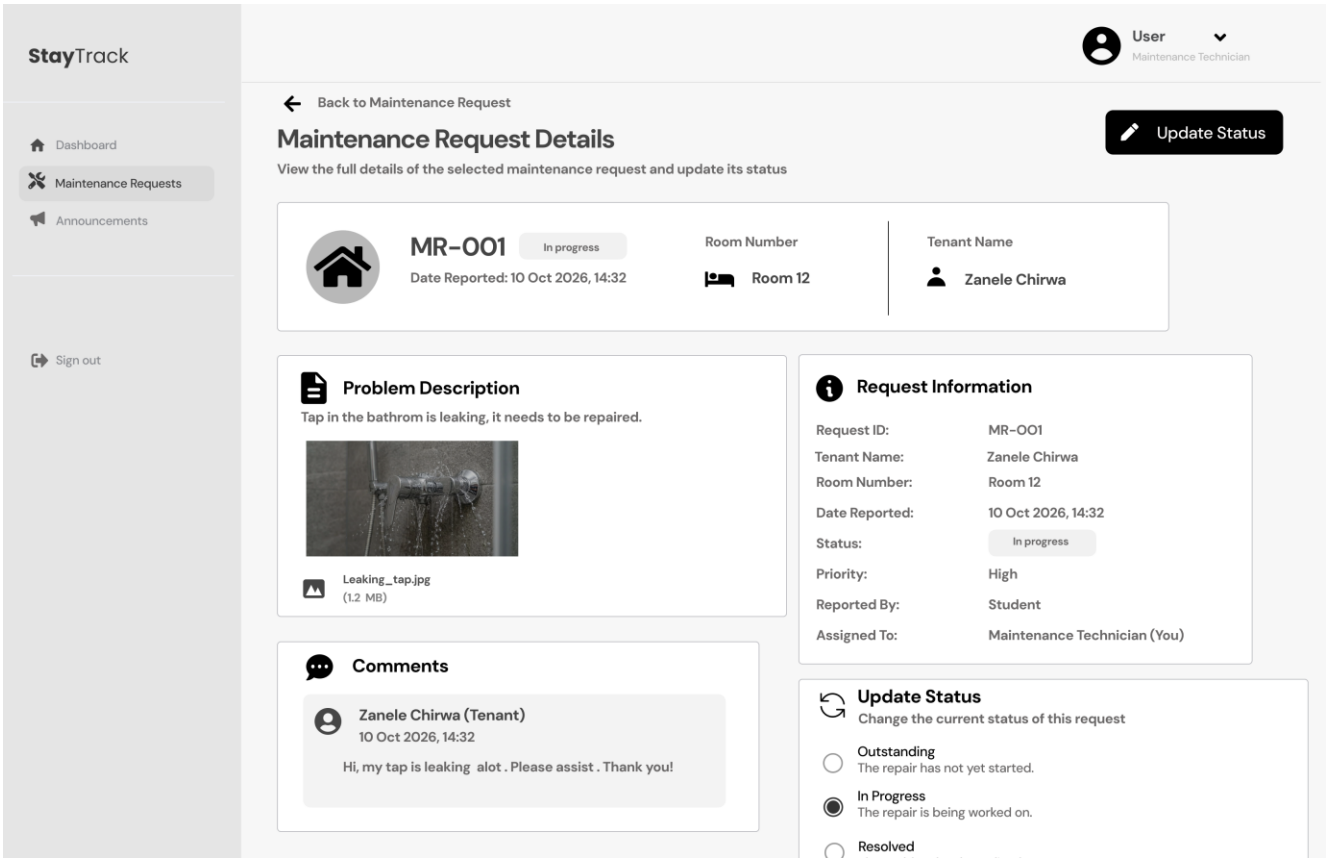
payment screen



Monthly report screen



Announcement screen



Detailed maintenance request screen

13.5.3 Navigation Between Screens

Flutter mobile application

Starting screen	User action	Next screen
Sign in / Sign up	Enters email and requests an OTP	OTP Verification
OTP Verification	Enters the correct OTP	Home
Home	Selects Upload Proof of Payment	Upload Proof of Payment
Upload Proof of Payment	Submits the proof	Confirmation, then My Payments
Home	Selects Report an Issue	Submit Maintenance Request
Submit Maintenance Request	Submits the request	Confirmation, then Maintenance Requests
Maintenance Requests	Selects a request	Request Detail
My Payments	Selects a payment	Payment Detail

ASP.NET web application

Starting page	User action	Next page
Login (OTP)	Management enters the correct OTP	Dashboard
Login (OTP)	Technician enters the correct OTP	My Assigned Requests
Dashboard	Selects Payments	Payments
Payments	Selects a pending payment and approves or rejects it	Payments (updated list)
Dashboard	Selects Monthly Report	Monthly Report
Monthly Report	Chooses a month and generates the report	Report shown and available to download
Maintenance Management	Assigns a request to a technician	Maintenance Management (updated list)
My Assigned Requests	Selects a request	Request Detail (technician)
Request Detail (technician)	Marks the job as completed	My Assigned Requests

13.5.4 Interface and ERD Check

Every item of information captured or displayed by the interfaces is supported by a table and field in the ERD.

Interface information	Table	Field
Student name, email, phone	`profiles`	full_name, email, phone
User role	`role`	role_name
Student number	`tenant`	student_number
Payment reference	`tenant`	payment_reference
Rental amount	`tenant`	rental_amount
Rental period	`tenant`	lease_start, lease_end
Room number and property name	room, property	room_number, name
Payment amount, date, month paid for and status	`payment`	amount, payment_date, period_month, status

Proof-of-payment file	payment	proof_url
Who verified a payment and when	payment	verified_by, verified_at
Maintenance request title and description	maintenance_request	title, description
Maintenance photo	maintenance_request	photo_url
Maintenance status and dates	maintenance_request	status, date_submitted, date_completed
Assigned technician	maintenance_request	technician_id
Announcement title, message and date	announcement	title, message, created_at

13.6 Design Alignment

The table links each requirement to the use case, sequence diagram, ERD element and interface that support it, to show that the designs describe the same system. A dash means the process is not shown in a sequence diagram; the assessment requires at least two.

Requirement	Use case	Sequence diagram	ERD element	Interface
FR1, FR2	View Payment Reference & History	—	tenant.payment_reference, payment	My Payments (Flutter)
FR12	Upload Proof of Payment	—	payment.proof_url, payment.status	Upload Proof of Payment (Flutter)
FR13	Verify Payment	Sequence Diagram 2	payment.status, verified_by, verified_at	Payments (ASP.NET)
FR5	Generate Monthly Report	—	payment.period_month, payment.status	Monthly Report (ASP.NET)
FR6, FR14	Submit Maintenance Request; Attach Photo	Sequence Diagram 1	maintenance_request	Submit Maintenance Request (Flutter)
FR7	View Maintenance Request Status	—	maintenance_request.status	Maintenance Requests (Flutter)
FR15	Assign Maintenance Request	—	maintenance_request.technician_id	Maintenance Management (ASP.NET)
FR16	View Assigned Maintenance Requests; Approve Completed Job	—	maintenance_request.status, date_completed	My Assigned Requests (ASP.NET)
FR10	Manage Tenant Records	—	tenant, room, property, profiles	Tenant Management (ASP.NET)
FR9	View Outstanding Payments & Tasks Dashboard	—	payment, maintenance_request	Dashboard (ASP.NET)
FR11	Post Announcement; View Announcements	—	announcement	Post Announcement (ASP.NET); Announcements (Flutter)

NFR1	Login	Sequence Diagram 3 (if included)	profiles, role	Sign in, OTP Verification (Flutter); Login (ASP.NET)

13.7 Stakeholder Feedback on the Designs

The group showed the important designs to the stakeholder and recorded her feedback.

Item	Details
Date of feedback	13/10/2026
Designs shown	Mobile wireframes and webapp wireframes
Stakeholder comments	Pleased with the current designs
Changes requested	none
Changes made	none
Evidence available	Email confirmation

Changes made to the designs after the feedback and the alignment check are recorded below.

Design artefact	Change made	Reason
Architecture	none	Stakeholder is pleased with the overall design
Use case diagram	none	Stakeholder is pleased with the overall design
Sequence diagrams	none	Stakeholder is pleased with the overall design
ERD	none	Stakeholder is pleased with the overall design
Flutter interface	none	Stakeholder is pleased with the overall design
ASP.NET interface	none	Stakeholder is pleased with the overall design

Gladys Belle <Belle.GN@ufs.ac.za>
To: Fidelis Belle <fidelisbelle21@gmail.com>

13 October 2026 at 19:37

Dear Fidelis and Team,

I must say that I am truly delighted with how beautifully the initial project design has come together. I am very proud of you all and the excellent work you have done so far.

Everything looks good from my side at this stage, so please feel free to proceed with the next phase of the project.

Well done, everyone! Keep up the excellent work. Your efforts are greatly appreciated.

Kind regards,
Gladys

From: Fidelis Belle <fidelisbelle21@gmail.com>
Sent: Tuesday, October 13, 2026 18:57
To: Gladys Belle <Belle.GN@ufs.ac.za>
Subject: Please review the StayTrack designs and give us your feedback
[Quoted text hidden]

E-mail messages exchanged with the University of the Free State adhere to terms detailed at http://www.ufs.ac.za/disclaimer for our disclaimer and at https://www.ufs.ac.za/privacy-notice/ for our privacy policy; all personal information provided will be treated as confidential.

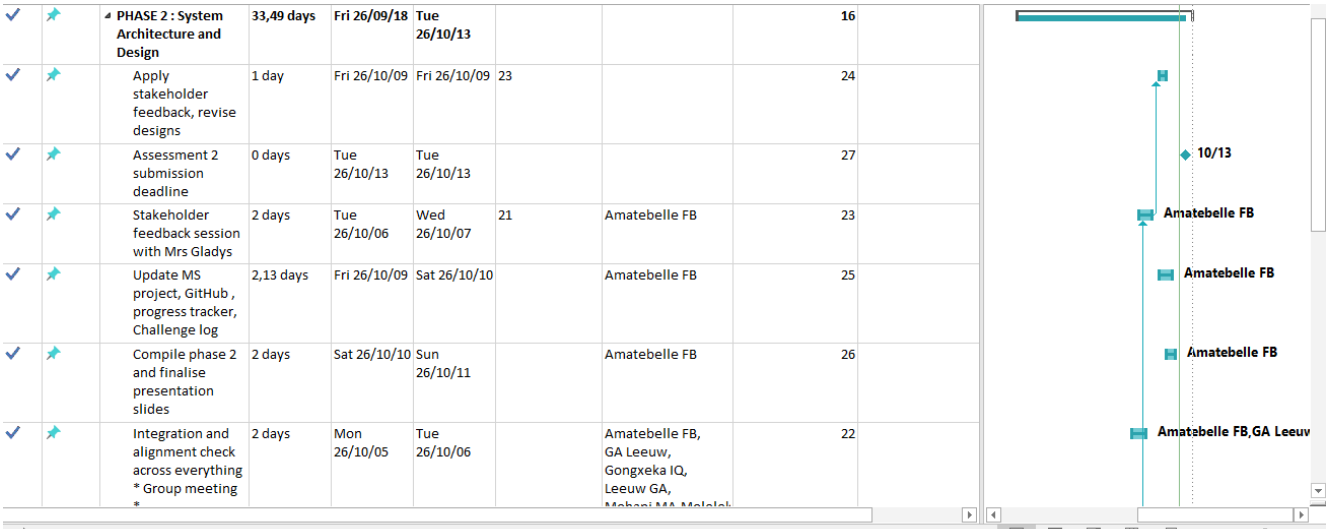
13.8 Original Design Source Links

Each design below can be opened and edited at its original source.

Design	Tool used	Source link or editable file name
System architecture diagram	Visily	system
Use case diagram	Visily	use case diagram link
Sequence diagram 1	Lucidchart	sequence diagram flutter
Sequence diagram 2	Lucidchart	sequence diagram asp.net
Entity relationship diagram	dbdiagram.io	enitiy relationship link
Flutter wireframes	Figma	flutter wire frames
ASP.NET wireframes	Figma	ASP.NET wireframes

13.9 Project Management Evidence

Microsoft Project. The updated schedule shows the completed Phase 2 design activities, the planned Flutter, ASP.NET and Supabase development and integration, testing and corrections, and final presentation preparation, with start and finish dates, durations, dependencies, assigned members, milestones and percentage complete. The original editable file is submitted with this SRS.



	Task Name	Duration	Start	Finish	Predecessors	Resource Names	ID
	PHASE 2 : System Architecture and Design	**33,49 days**	**Fri 26/09/18**	**Tue 26/10/13**			**16**
	Apply stakeholder feedback, revise designs	1 day	Fri 26/10/09	Fri 26/10/09	23		24
	Assessment 2 submission deadline	0 days	Tue 26/10/13	Tue 26/10/13			27
	Stakeholder feedback session with Mrs Gladys	2 days	Tue 26/10/06	Wed 26/10/07	21	Amatebelle FB	23
	Update MS project, GitHub , progress tracker, Challenge log	2,13 days	Fri 26/10/09	Sat 26/10/10		Amatebelle FB	25
	Compile phase 2 and finalise presentation slides	2 days	Sat 26/10/10	Sun 26/10/11		Amatebelle FB	26
	Integration and alignment check across everything * Group meeting *	2 days	Mon 26/10/05	Tue 26/10/06		Amatebelle FB, GA Leeuw, Gongxeka IQ, Leeuw GA,	22

	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names	ID
18		Use case diagram design	7,87 days	Fri 26/09/18	Wed 26/09/23		Amatebelle FB, Mohapi MA	18
20		Design the system database	8,87 days	Thu 26/09/24	Wed 26/09/30		Amatebelle FB, Molelekeng KP	20
21		design the flutter & asp.net wireframes	6,87 days	Thu 26/10/01	Mon 26/10/05	18,20	GA Leeuw, Gongxeka IQ, Mohapi MA,Seleke	21
17		System Architecture diagram design	5,87 days	Fri 26/09/18	Mon 26/09/21		GA Leeuw, Tshitangano TD	17
19		develop the sequence diagram	7,87 days	Thu 26/09/24	Tue 26/09/29	18	Gongxeka IQ, Seleke TM	19
28		**PHASE 3: Development, Intergration and Testing**						**28**

Figure 27: Updated Microsoft Project schedule at the end of Phase 2

GitHub. The group repository is available at StayTrack github link It contains the Flutter and ASP.NET project files and a docs folder with the project documentation and design links. No passwords, API keys or confidential stakeholder information are stored in it.

Progress Tracker. The updated ITC327W Project Progress Tracker is submitted with this SRS as an Excel workbook. It records the Phase 2 tasks, individual contributions, challenges and actions, and the Phase 2 group reflection.

13.10 Next Steps for Phase 3

Development will start with the Supabase backend: the tables from the ERD, OTP authentication, storage buckets and access rules. The group will then build the Flutter and ASP.NET screens designed above, starting with the Must Have payment and maintenance functions, integrate both applications with the shared backend, and test that data created in one application appears correctly in the other. The group will update the project schedule, GitHub repository and Progress Tracker as work progresses.

Appendices

Appendix A — Stakeholder Interview Questions:

Stakeholder Interview Questions

Current Process

1. Can you explain how you currently manage students, rooms and accommodation information?

2. Can you walk us through the current process you follow when a student makes an accommodation payment, from the payment being made until it is recorded and confirmed?

Problems Experienced

3. Which parts of managing the accommodation are the most time-consuming or difficult for you, and why?

4. What problems, mistakes or delays tend to occur when using your current methods, and how do these affect your day-to-day work?

Intended Users

5. Who are the different people who would need to use an accommodation-management system, and what would each person use it for?

6. What information or tasks would you expect each type of user to access or manage?

Desired Mobile/Web Functions

7. Based on the challenges you have described, what tasks or processes would you most like to make easier, faster or more efficient?

8. Which tasks do students or staff currently perform manually or through platforms such as WhatsApp, Excel or email that you would prefer to manage in one place?

Information, Reports, Access and Security

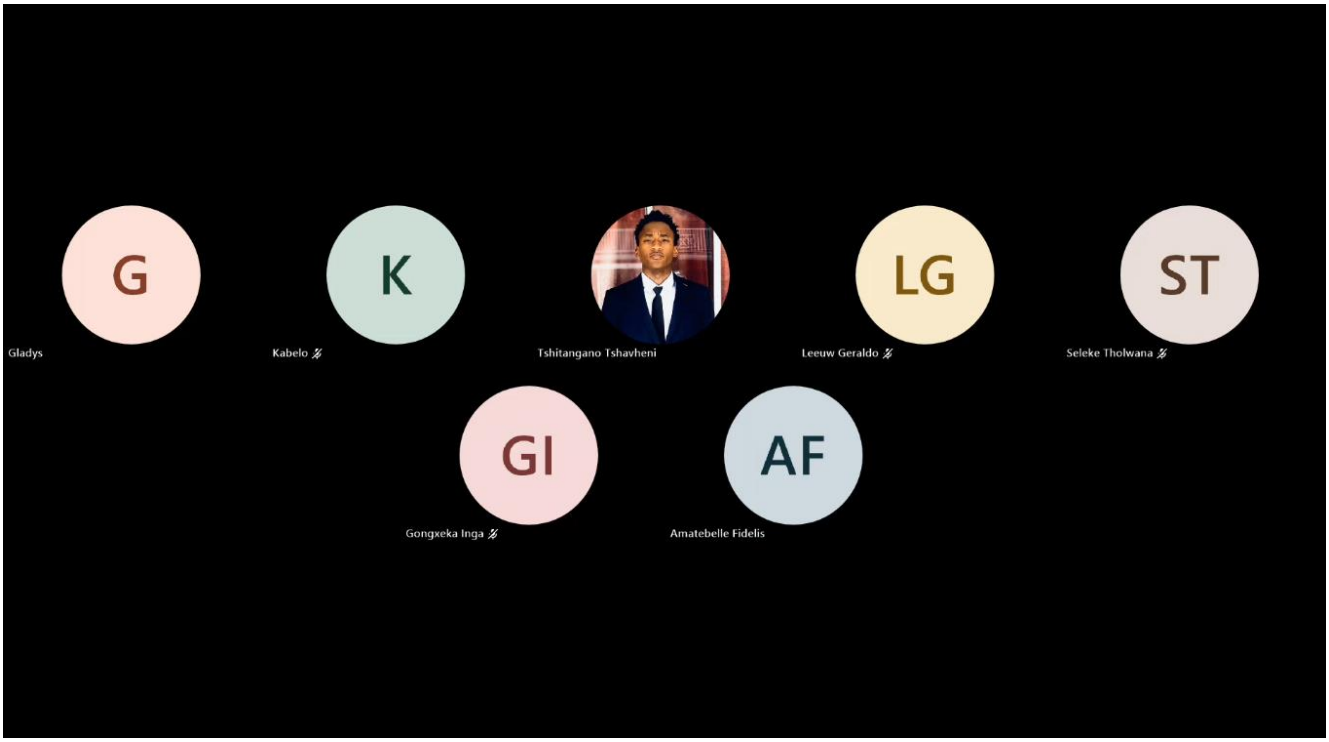
9. What information or reports do you currently need to check regularly when managing the accommodation, and what would you like to be able to see more easily?

10. How should access to student, payment and accommodation information be managed, and what security or privacy requirements would you consider important?

Appendix B — Stakeholder Responses / Interview Notes:

- Interview notes pdf link

Appendix C — Evidence of Stakeholder Engagement:



1st Interview meeting with the Skakeholder 08000900-20260824_90255_AM-Meeting Recording.mp4

Appendix D — Stakeholder Confirmation of Findings:

- ❑ PDF OF STAKEHOLDER FINDINGS